

== Algoritma Analizi ==

BLM3021 Mine ELİF KARSLIGİL & M. ArmaG GÜVEN SAN

Kaynaklar :

- 1) The Design & Analysis of Algorithms , A. Levitin
- 2) Algorithms : 4th Edition , R. Sedgewick , K. Wayne
<http://www.algs4.princeton.edu>

3 ödev : %15

2 Vize : %45

Proje : %10

Final : %30

** Programlama dili $\Rightarrow$ C dili (ödevlere göre değişebilir)

Algoritma Yer Karmaşıklığı

- $\propto$ değişkenler
- $\propto$ sabitler
- $\propto$ program komutları
- $\propto$ fonk. çağırma vb. sebeplerle yığın işlemleri
- $\propto$ rekürsif fonksiyonları yığını öldürabiliyor.

Algoritmaların Karşılaştırması

Doğru mu?

Hızlı mı?

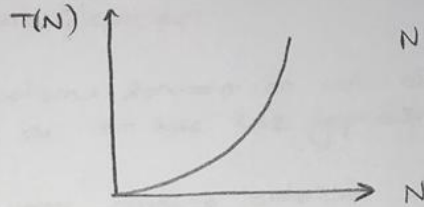
Az yer kaplıyor mu?

Algoritma Gölisme Zamanının Deneyisel Öölümü

3-Sum problemi : Verilen N farklı sayı ıginden toplamı $\emptyset$ eden üçölünün bulunması

$a[i]$	$a[j]$	$a[k]$	sum
-30	-10	40	$\emptyset$
-30	-10	50	10
-10	40	50	80
-10	40	-30	$\emptyset$

```
for(i=0; i<N; i++)
  for(j=i+1; j<N; j++)
    for(k=j+1; k<N; k++)
      if(a[i]+a[j]+a[k]==0)
        Count++;
```

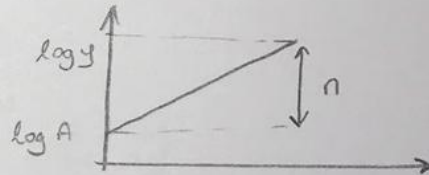


N 'in gölisme zamanı $T(N)$ 'e göre

Log-log plot: N 'nin $T(N)$ 'e göre logaritmik değışimi çizilir.

$y = A * x^n$ ise iki tarafın logaritması alınır

$$\begin{aligned} \log y &= \log(A * x^n) \\ &= \log(A) + \log(x^n) \\ &= \log(A) + n \cdot \log x \end{aligned}$$



Deneyisel Gölümelerde Gölisme Zamanını Etkileyenler:

Donanım : CPU, bellek, cache
Yazılım : Derleyici
Sistem : OS, diğer uygulamalar vb.

) Sistemle bağlı

Bağımsız etkenler

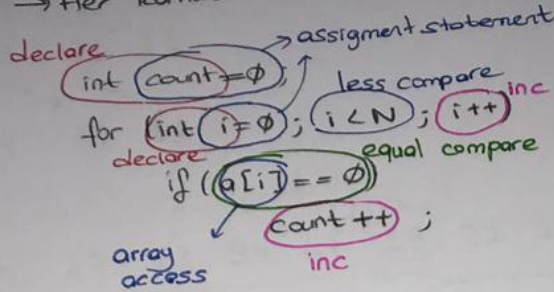
× Algoritma
× Data (input)

! Deneyisel göölümelerde her türlü donanım ve yazılım olanaklarının aynı olması gerekir.

Galışma Zamanının Matematik Modeli

* Bütün komutların çalışma zamanlarının toplamı.

→ Her komutun çalışma zamanı aynı varsayılırsa



Her kodu tek tek toplamak gerekiyor.

Galışma Zamanının Teorik Ölölmesi

Ana İşlem: Algoritmanın çalışma zamanını en çok alan, en önemli işi belirle. ve bu işin kaç kez yapıldığını değerlendir.

Asimptotik Analiz: Giris verisi sonsuza giderken algoritmanın performans analizi nasıl yapılır?

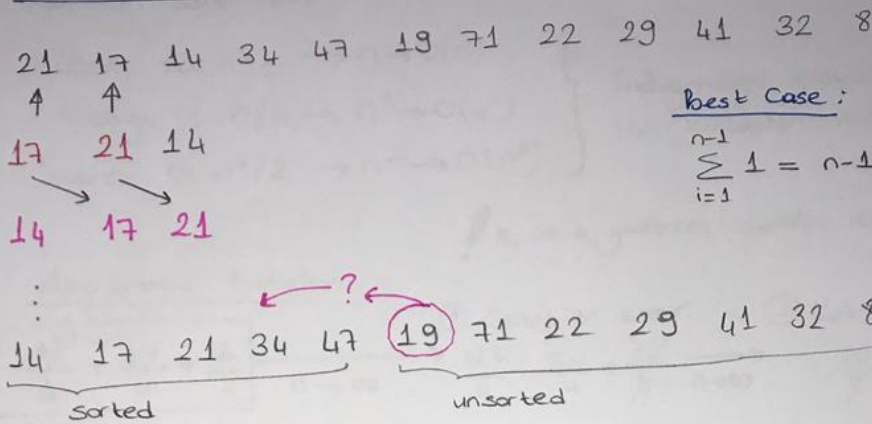
Büyüme Derecesi: N büyüdükçe algoritmanın çalışma hızı nasıl değişiyor?

Hızı etkileyenler:

- 1) Girisdeki eleman sayısı
- 2) Bu elemanların hangi düzende olduğu

! Bu etkenleri ölmek için best, average ve worst senaryolarını hesaplamak gerekir.

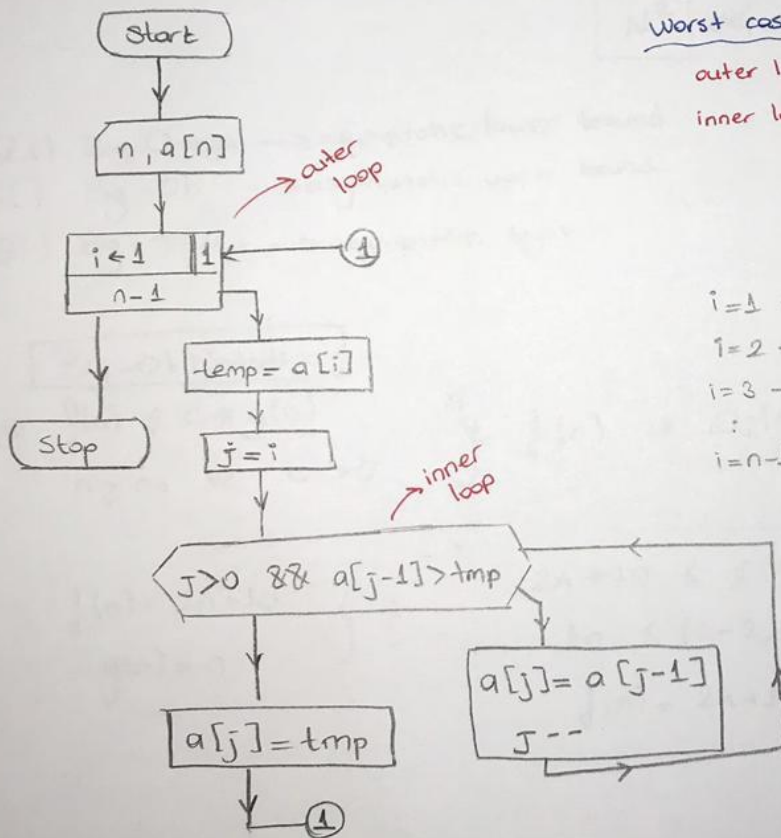
Insertion Sort



Best Case:

$$\sum_{i=1}^{n-1} 1 = n-1 \sim \boxed{n}$$

best



Worst case:

outer loop: 1 ... (n-1)

inner loop: $\left. \begin{matrix} j=1 \\ j=2 \\ j=3 \\ \vdots \\ j=n-1 \end{matrix} \right\} n-1$

i=1 → j=1

i=2 → j=0, 1

i=3 → j=0, 1, 2

...

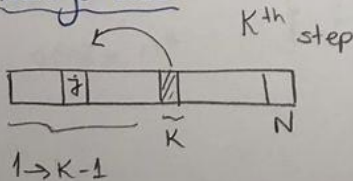
i=n-1 → j=0, 1, 2, ..., (n-2)

$$\sum_{i=1}^{n-1} \sum_{j=1}^{i-1} 1 = \sum_{i=1}^{n-1} i$$

$$= \frac{n(n-1)}{2} \sim \boxed{\frac{n^2}{2}}$$

Worst

Average Case:



$$\frac{1}{K} \cdot \sum_{j=1}^K (K-j+1) = \frac{1}{K} \left(K^2 - \frac{K(K-1)}{2} + K \right)$$

$$= \frac{2K^2 - K^2 - K + 2K}{2K} = \frac{K+1}{2}$$

$$\sum_{k=2}^N \frac{K+1}{2} = \frac{1}{2} \cdot \frac{(N+1)(N+2)}{2} = \boxed{\frac{1}{4} [N^2 + 3N + 2]}$$

Average

Insertion sort için

$$\left. \begin{array}{l} \text{Best} \approx n-1 \rightarrow n \rightarrow O(n) \\ \text{Average} \approx n^2/4 \rightarrow n^2 \rightarrow O(n^2) \\ \text{Worst} \approx n^2/2 \rightarrow n^2 \rightarrow O(n^2) \end{array} \right\} \text{İndirgenliğini göstermek için kullanılan notasyon}$$

! n, ∞ 'a gidenken sabitler bizi ilgilendirmez.

Asymptotic Notation

$$\boxed{\frac{N^2}{4} + \frac{3N}{4} + \frac{2}{4}} \xrightarrow{n \rightarrow \infty} \frac{N^2}{4} + \frac{3N}{4} + \frac{2}{4} \xrightarrow{n \rightarrow \infty} \frac{N^2}{4} + \frac{3N}{4} \xrightarrow{n \rightarrow \infty} \frac{N^2}{4}$$

① Sabitler elenir. ② Kot sayılar elenir.

③ En yüksek dereceli terim alınır.

N^2 ← $N^2 + \cancel{N}$

- $\Omega()$ Big-Omega $\rightarrow$ asymptotic lower bound
 $O()$ Big-Oh $\rightarrow$ asymptotic upper bound
 $\Theta()$ Big-Theta $\rightarrow$ asymptotic tight

Big-Oh Notation

$$\textcircled{\times} f(n) \leq c * g(n) \quad \left. \begin{array}{l} n \geq n_0 \text{ ve } c > 0 \end{array} \right\} f(n) \text{ is } O(g(n))$$

$$\left. \begin{array}{l} f(n) = 2n+10 \\ g(n) = n \end{array} \right\} ?$$

$$\begin{array}{l} 2n+10 \leq c * n \quad n \geq n_0 \\ 10 \leq (c-2) * n \\ f(n) = 2n+10 \rightarrow O(n) \end{array} \quad \begin{array}{l} c=3 \quad n=10 \checkmark \end{array}$$

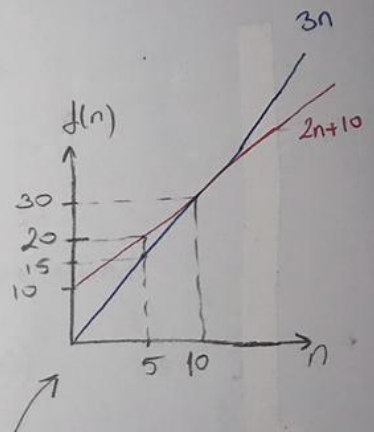
$\uparrow$
 $g(n)$

$$\left. \begin{array}{l} f(n) = 2n+10 \\ g(n) = n^2 \end{array} \right\} ?$$

$$\begin{array}{l} 2n+10 \leq c * n^2 \quad n \geq n_0 \quad c > 0 \\ cn^2 - 2n \geq 10 \quad n=1 \quad c=100 \checkmark \end{array}$$

$$f(n) = 2n+10 \rightarrow O(n^2)$$

$\uparrow$
 $g(n)$



Big-Omega Notation

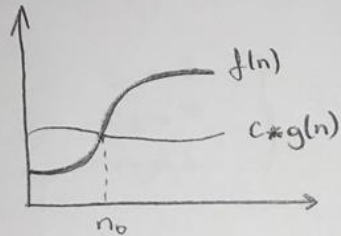
asymptotic lower bound

$$\{ f(n) \in \Omega(g(n)) \mid \text{if } c > 0, n_0 > 0, n \geq n_0 \} \Rightarrow f(n) \geq c * g(n)$$

Insertion Sort $\rightarrow$ best case: $\Omega(n)$

Sequential Search $\rightarrow$ best case: $\Omega(1)$

$$f(n) = 5n^2 \rightarrow \Omega(n^2)$$



$$5n^2 \geq c * n^2$$

$$c > 0 \quad n > n_0 \quad n_0 > 0$$

$$c = 1 \quad n_0 = 1 \Rightarrow (5 - c)n^2 \geq 0 \quad \checkmark$$

$$f(n) = 5n^2 \rightarrow \Omega(n)$$

$$5n^2 \geq c * n \Rightarrow n(5 - c * n) \geq 0$$

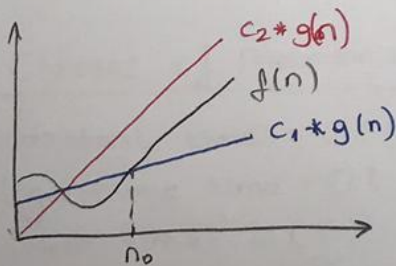
$$c > 0 \quad n > n_0 \quad n_0 > 0$$

Big-Theta Notation

tight bound

$$0 \leq c_1 * g(n) \leq f(n) \leq c_2 * g(n) \quad \{ f(n) = \Theta(g(n)) \}$$

$$c_1 \leq c_2, n_0 > 0 \quad n > n_0$$



$$\left. \begin{array}{l} f(n) \in \Omega(g(n)) \\ f(n) \in O(g(n)) \end{array} \right\} \Omega(g(n)) = O(g(n))$$

$$f(n) = 2n + 5 \rightarrow \Theta(n^2) \text{ diyemem çünkü } \Omega(n^2) \text{ değildir.}$$

$$\rightarrow \Omega(1) \text{ diyemem çünkü } \Theta(1) \text{ değildir.}$$

$$f(n) = 2n+5 \in ? \Theta(n)$$

$$\textcircled{1} 2n+5 \in ? \Omega(n) \rightarrow 2n+5 \geq c*n$$

$$5 \geq (c-2)n$$

$$n_0=1 \quad c_1=3 \quad \checkmark\checkmark$$

$$\textcircled{2} 2n+5 \in ? O(n) \rightarrow 2n+5 \leq c*n$$

$$5 \leq (\frac{c}{2}-2)n$$

$$n_0=1 \quad c_2=7 \quad \checkmark\checkmark$$

$$\left. \begin{matrix} c_1=3 \\ c_2=7 \\ n_0=1 \end{matrix} \right\} c_1*n \leq 2n+5 \leq c_2*n \rightarrow f(n) \in \Theta(n)$$

Insertion Sort :

$$\text{worst case } \sim n^2 \rightarrow \Theta(n^2)$$

$$\text{best case } \sim n \rightarrow \Theta(n)$$

$$\left. \begin{matrix} \text{insertion sort} \\ O(n^2) \rightarrow \text{max } n^2 \text{ olabilir} \\ \Omega(n) \rightarrow \text{min } n \text{ olabilir.} \end{matrix} \right\}$$

~~$\Theta(n)$~~

Merge Sort :

$$\left. \begin{matrix} \text{Best case } \sim n \log n \\ \text{worst case } \sim n \log n \end{matrix} \right\} \Theta(n \log n)$$

Classes of Algorithms :

	example
constant time : $O(1)$	$a=b+c$
logarithmic time : $O(\log n)$	binary search
linear time : $O(n)$	1 Loop
linearithmic time : $O(n \log n)$	merge sort
quadratic time : $O(n^2)$	double loop
cubic : $O(n^3)$	triple loop
exponential : $O(2^n)$	exhaustive search

Useful Formulas for the Analysis of Algorithms

Properties of Logarithms

1. $\log x^y = y \cdot \log x$
2. $\log xy = \log x + \log y$
3. $\log \frac{x}{y} = \log x - \log y$
4. $\log_a x = \log_a b \log_b x$
5. $a^{\log_b x} = x^{\log_b a}$

Combinatorics

1. Number of permutations of an n -element set:
 $P(n) = n!$
2. Number of k -combinations of n -elements set:
 $C(n, k) = \frac{n!}{k!(n-k)!}$
3. Number of subsets of an n -elements set: 2^n

Important Summation Formulas

1. $\sum_{i=l}^u 1 = \underbrace{1+1+\dots+1}_{u-l+1} = u-l+1 \quad (1 \leq u)$
2. $\sum_{i=1}^n i = 1+2+\dots+n = \frac{n(n+1)}{2} \approx \frac{1}{2} n^2$
3. $\sum_{i=1}^n i^2 = 1^2+2^2+\dots+n^2 = \frac{n(n+1)(2n+1)}{6} \approx \frac{1}{3} n^3$
4. $\sum_{i=1}^n i^k = 1^k+2^k+\dots+n^k \approx \frac{1}{k+1} n^{k+1}$
5. $\sum_{i=0}^n a^i = 1+a^1+a^2+\dots+a^n = \frac{a^{n+1}-1}{a-1} \quad (a \neq 1)$
6. $\sum_{i=1}^n i \cdot 2^i = 1*2 + 2*2^2 + \dots + n \cdot 2^n = (n-1)2^{n+1} + 2$
7. $\sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \approx \ln n + \gamma \quad (\gamma = 0.5772)$
8. $\sum_{i=1}^n \lg i \approx n \lg n$

$$\boxed{\sum_{i=0}^n 2^i = 2^{n+1} - 1}$$

Sum Manipulation Rules

1. $\sum_{i=l}^u c \cdot a_i = c \cdot \sum_{i=l}^u a_i$
2. $\sum_{i=l}^u (a_i \mp b_i) = \sum_{i=l}^u a_i \mp \sum_{i=l}^u b_i$
3. $\sum_{i=l}^u a_i = \sum_{i=l}^m a_i + \sum_{i=m+1}^u a_i$ where $(l \leq m < u)$

Floor and Ceiling Formulas

- $\lfloor 3.8 \rfloor = 3$
 $\lfloor -3.8 \rfloor = -4$
 $\lceil 3.8 \rceil = 4$
 $\lceil -3.8 \rceil = -3$
2. $\lfloor x+n \rfloor = \lfloor x \rfloor + n$ and $\lceil x+n \rceil = \lceil x \rceil + n \Rightarrow$ for real x and integer n
3. $\lfloor n/2 \rfloor + \lceil n/2 \rceil = n$
4. $\lceil \log(n+1) \rceil = \lfloor \log n \rfloor + 1$

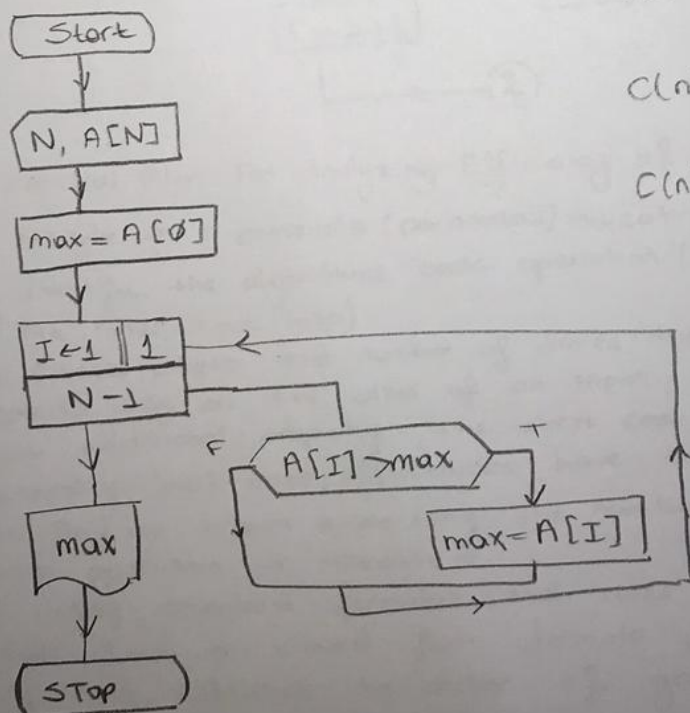
Mathematical Analysis of Non-Recursive Algorithms

Two operations: $A[I] > \text{max}$ (Basic operation)
 $\text{max} = A[I]$

$C(n)$: the number of times this comparison is executed

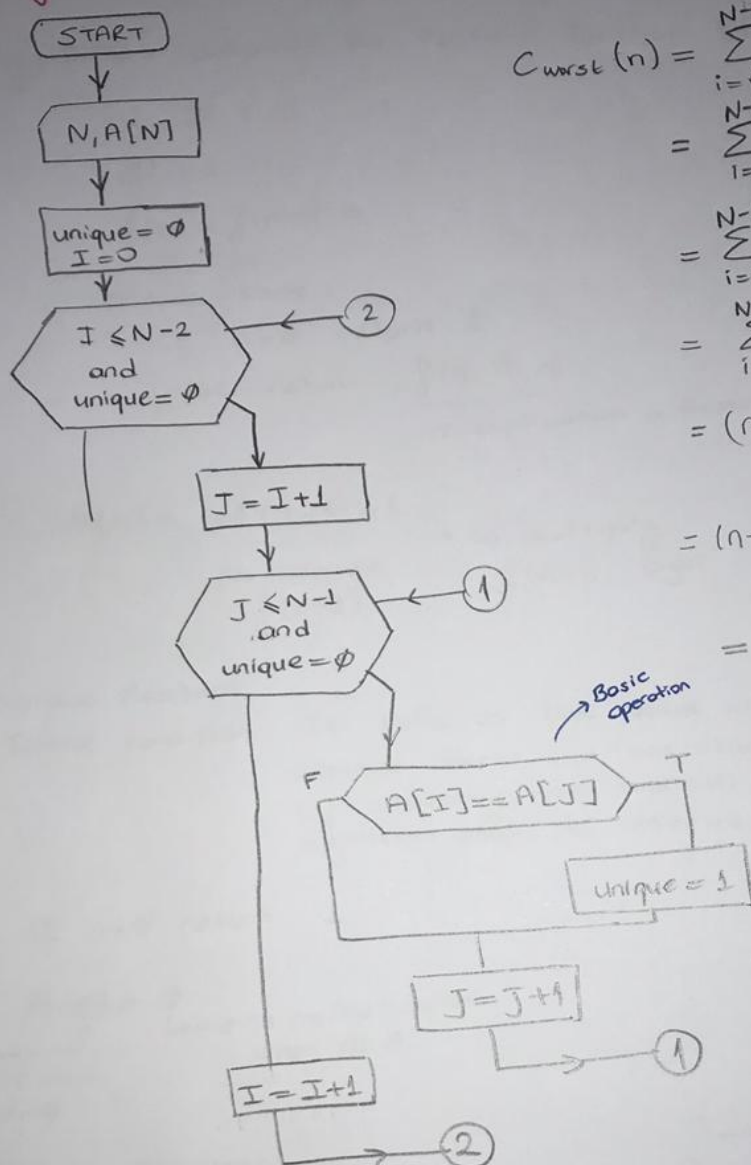
$$C(n) = \sum_{i=1}^{n-1} 1 = n-1 \approx n$$

$\Theta(n)$



Algorithms Element Unique Problem

10



$$\begin{aligned}
 C_{\text{worst}}(n) &= \sum_{i=0}^{N-2} \sum_{j=i+1}^{N-1} 1 \\
 &= \sum_{i=0}^{N-2} ((n-1) - (i+1) + 1) \\
 &= \sum_{i=0}^{N-2} (n-i-1) \\
 &= \sum_{i=0}^{N-2} (n-1) - \sum_{i=0}^{N-2} i \\
 &= (n-1) \cdot \sum_{i=0}^{N-2} 1 - \frac{(N-2)(N-1)}{2} \\
 &= (n-1)^2 - \frac{(n-2) \cdot (n-1)}{2} \\
 &= \frac{(n-1)n}{2} \approx \frac{1}{2} n^2 \in \Theta(n^2)
 \end{aligned}$$

General Plan For Analyzing Efficiency of Non Recursive Algorithms

1. Decide on a parameter (parameters) indicating input's size
2. Identify the algorithm's basic operation: (As a rule, it is located in its inner-most loop)
3. Check whether the number of times the basic operation is executed depends only on the size of an input. If it also depends on some additional property, the worst case, average-case and if necessary best-case, efficiencies have to be investigated separately.
4. Set up a sum expressing the number of times the algorithm's basic operation is executed.
5. Using standard formulas and rules of sum manipulation, either find a closed-form formula for the count or, at the very least, establish its order of growth.

Mathematical Analysis of Recursive Algorithms

Algorithms: computing the Factorial Function $F(n) = n!$

$$n! = 1.2.3. \dots n$$

$$0! = 1$$

$$f(n) = f(n-1) \cdot n$$

pseudo-code:

if $n=0$ return 1

else return $f(n-1) \cdot n$

multiplication = Basic operation!

$$M(n) = \underbrace{M(n-1) + 1}_{\text{to compute } f(n-1)} \rightarrow \text{to multiply } f(n-1) \text{ by } n$$

Recursive Relations

Initial condition: It tells us the value with which the sequence starts. We can obtain this value by inspecting the condition that makes the algorithm stop its recursive calls.

if $n=0$ return 1

$$M(0) = 0 \rightarrow \text{no multiplication when } n=0$$

the calls stop when $n=0$

Recurrence Relations

$$M(n) = M(n-1) + 1 \text{ for } n > 0$$

$$M(0) = 0$$

Method of Backward Substitution

$$\begin{aligned} M(n) &= M(n-1) + 1 && \text{substitute } M(n-1) = M(n-2) + 1 \\ &= [M(n-2) + 1] + 1 && \text{substitute } M(n-2) = M(n-3) + 1 \\ &= [M(n-3) + 1] + 2 = M(n-3) + 3 \\ &\vdots \end{aligned}$$

$$M(n) = M(n-i) + i$$

To take advantage of initial condition,

$$M(n) = M(n-1) + 1 = M(n-i) + i = \dots = M(n-n) + n = n //$$

$$\boxed{\begin{array}{l} n = \phi, i = n \\ M(\phi) + \phi = \phi \end{array}}$$

A General Plan for Analyzing Efficiency of Recursive Algorithms

İlk 3 madde recursive fonksiyonlar ile aynı

4. Set up a recurrence relation with an appropriate initial condition, for the number of times the basic operation is executed.
5. Solve the recurrence or at least ascertain the order of growth of its solution

Ex/

Input: A positive decimal integer

The number of binary digits in binary's representation

if $n=1$ return 1

else return BinaryDigit $\lfloor n/2 \rfloor + 1$

$A(n)$: The number of Additions

$$\boxed{A(n) = A(\lfloor n/2 \rfloor) + 1}$$

Initial Condition:

$$n=1 \quad A(1) = \phi$$

$$n=2^k \rightarrow A(2^k) = A(2^{k-1}) + 1 \quad A(2^0) = \phi$$

Backward Substitution
 devamı

Backward substitution

$$A(2^k) = A(2^{k-1}) + 1 \quad \text{substitute } A(2^{k-1}) = A(2^{k-2}) + 1$$

$$A(2^k) = [A(2^{k-2}) + 1] + 1 \quad \text{substitute } A(2^{k-2}) = A(2^{k-3}) + 1$$

$$\rightarrow A(2^{k-2}) + 2$$

$$A(2^k) = [A(2^{k-3}) + 1] + 2 = A(2^{k-3}) + 3$$

$$\vdots$$

$$\vdots$$

$$= A(2^{k-i}) + i$$

$$\vdots$$

$$= A(2^{k-k}) + k$$

$$= A(2^0) + k = k$$

$$\underbrace{A(1)} = 0$$

$$n = 2^k \Rightarrow k = \log_2 n$$

$$A(n) = \log_2 n \in \Theta(\log n)$$

Binary Search Algorithm

Input : $A[0 \dots n-1]$ sorted array

Output : Index or -1

Pseudo Code :

$l \leftarrow 0$; $r \leftarrow n-1$

while ($l \leq r$) do

$m \leftarrow \lfloor (l+r)/2 \rfloor$

if $K = A[m]$ return m

else if $K < A[m]$ $r \leftarrow m-1$

else $l \leftarrow m+1$

return -1

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
Value	3	14	27	31	39	42	55	70	76	81	85	93	98
Iteration 1	l						m						r
Iteration 2							l			m			
Iteration 3							l	m	r				

$\rightarrow$ c = compare
 w = worst case
 $C_w(n) = C_w(\lfloor n/2 \rfloor) + 1$, for $n > 1$

Initial Condition:

$C_w(1) = 1$

$n = 2^k \rightarrow$ problem 2'ye bölünerek devam ettiği için.

Backward Substitution:

$C_w(2^k) = k + 1 = \log_2 n + 1$ (Bir önceki sayfada aynı var)

Verify by Substitution that function $C_w(n) = \lfloor \log_2 n \rfloor + 1$ indeed satisfies equation for any positive even number

$n = 2^i, i > 0$

$C_w(n) = \lfloor \log_2 n \rfloor + 1 = \lfloor \log_2 2^i \rfloor + 1 = \lfloor \log_2 2 + \log_2 i \rfloor + 1$
 $= \lfloor 1 + \log_2 i \rfloor + 1 = \log_2 i + 2$

$C_w(\lfloor n/2 \rfloor) + 1 = C_w(\lfloor 2^i/2 \rfloor) + 1 = C_w(i) + 1$
 $= (\lfloor \log_2 i \rfloor + 1) + 1 = \lfloor \log_2 i \rfloor + 2$

The worst-case efficiency of Binary Search: $\Theta(\log n)$

$$C_{\text{avg}}(n) \approx \log_2 n \rightarrow C_{\text{avg}}^{\text{yes}} \approx \log_2 n - 1$$

$$C_{\text{avg}}^{\text{no}} \approx \log_2(n+1)$$

$$\lfloor \log_2 10^3 \rfloor + 1 = 10$$

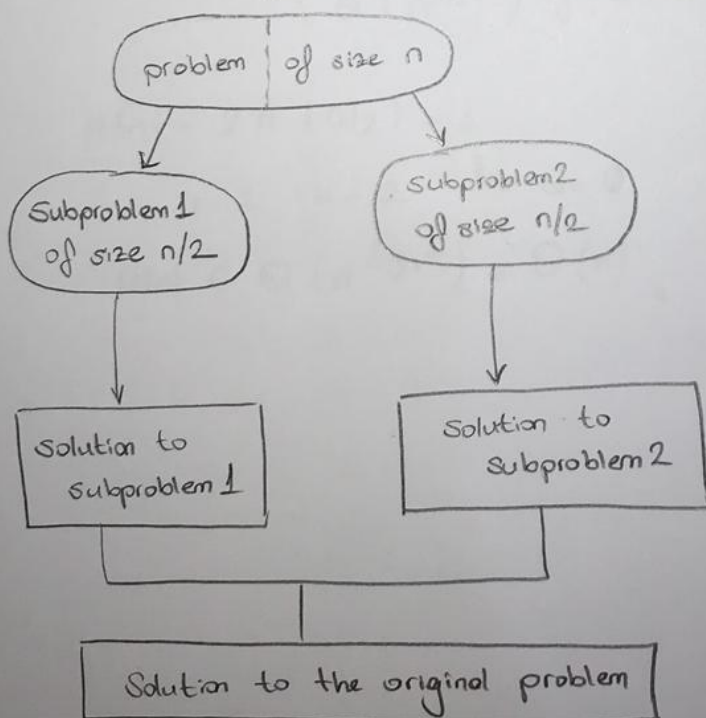
$$\lfloor \log_2 10^6 \rfloor + 1 = 20$$

Binary search'ü recursive gözetmeye
gölg

Divide and Conquer Algorithms

They work according to the following general plan.

1. A problem's instance is divided into several smaller instances of the same problem, ideally about the same size.
2. The smaller instances are solved (typically recursively though sometimes a different algorithm is employed when instances become small enough).
3. If necessary, the solutions obtained for the smaller instances are combined to get a solution to the original problem.



İsimlere yarımayacak
ise alt problemlere
ayrılmak zorunda
olunur.

Computing the sum of n

$$a_0 + a_1 + a_2 + \dots + a_{n-1} = (a_0 + a_1 + \dots + a_{\lfloor n/2 \rfloor - 1}) + (a_{\lfloor n/2 \rfloor} + \dots + a_{n-1})$$

Instance size: n

Subproblems: b

the subproblems that need to be solved: a

$$T(n) = a T(n/a) + f(n) \Rightarrow \text{Running Time} \left[\begin{array}{l} \text{General Divide and} \\ \text{Conquer Recurrence} \end{array} \right]$$

$\hookrightarrow f(n)$ is a function that accounts for the time spent on dividing the problem into smaller ones and combining their solutions.

Master Theorem

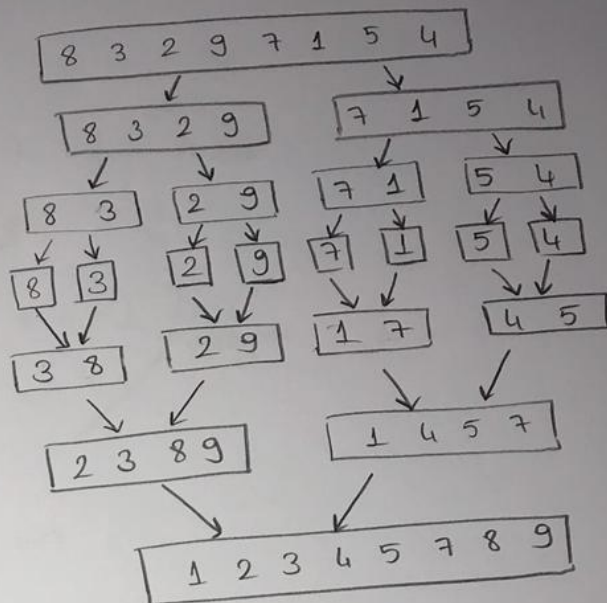
if $f(n) \in \Theta(n^d)$ where $d \geq 0$

$$T(n) \in \begin{cases} \Theta(n^d) & \text{if } a < b^d \\ \Theta(n^d \log n) & \text{if } a = b^d \\ \Theta(n^{\log_b a}) & \text{if } a > b^d \end{cases}$$

$$A(n) = 2A(n/2) + 1$$

$$a=2 \quad b=2 \quad \hookrightarrow d=0$$

$$A(n) \in \Theta(n^{\log_2 2}) = \Theta(n)$$

Merge Sort AlgorithmPseudo Code: $A[0 \dots n-1]$ if $(n > 1)$ copy $A[0 \dots \lfloor n/2 \rfloor - 1]$ to $B[0 \dots \lfloor n/2 \rfloor - 1]$ copy $A[\lfloor n/2 \rfloor \dots n-1]$ to $C[0 \dots \lfloor n/2 \rfloor - 1]$ mergeSort($B[0 \dots \lfloor n/2 \rfloor - 1]$)mergeSort($C[0 \dots \lfloor n/2 \rfloor - 1]$)merge(B, C, A)

$$C(n) = 2C(n/2) + C_{\text{merge}}(n) \quad n > 1$$

Initial Condition : $C(1) = 0$ Partition Algorithm:Merge($B[0 \dots p-1], C[0 \dots q-1], A[0 \dots p+q-1]$) $i \leftarrow 0; j \leftarrow 0; k \leftarrow 0$ while $i < p$ and $j < q$ doif $B[i] \leq C[j]$ | $A[k] \leftarrow B[i]; i \leftarrow i+1$

else

| $A[k] \leftarrow C[j]; j \leftarrow j+1$ | $k \leftarrow k+1$ if $i = p$ | copy $C[j \dots q-1]$ to $A[k \dots p+q-1]$

else

| copy $B[i \dots p-1]$ to $A[k \dots p+q-1]$

Recurrence Relation

$$C(n) = 2C(n/2) + C_{\text{merge}}(n) \quad \text{for } n > 1$$

$$\text{Initial Condition : } C(1) = \phi$$

$$C_{\text{worst}}(n) = 2C_{\text{worst}}(n/2) + n - 1$$

$n/2$
versiyonu

Backward Substitutions

$$C(n/2) = 2C(n/4) + n/2$$

$$C_{\text{worst}}(n) = 2[2C(n/4) + n/2] + n$$

$$= 4C(n/4) + 2n$$

$$= 8C(n/8) + 3n$$

$$= 2^i C(n/2^i) + i * n$$

$n = 2^i \rightarrow i = \log_2 n$

$$= n * \phi + \log_2 n * n$$

$\downarrow$ initial condition'a bağlı olarak

$$C_{\text{worst}}(n) \in \Theta(n \log_2 n)$$

! Principal Shortcoming : Linear amount of extra storage

Quick Sort Algorithm

Quick Sort divides the input's elements according to their value.

$A[0] \dots A[s-1]$ $A[s]$ $\underbrace{A[s+1] \dots A[n-1]}$

All are $\leq A[s]$ All are $\geq A[s]$

Obviously, after a partition has been achieved, $A[s]$ will be in its final position in the sorted array.

$\xrightarrow{\text{en büyük bul}}$ $\begin{array}{ccccccc} & & & i & & & \\ \boxed{5} & 3 & 1 & \boxed{9} & 8 & 2 & \boxed{4} & 7 \\ & & & j & & & & \end{array}$

Simplest Strategy
 $p = A[l]$

pseudo Code :

```

if  $l < r$ 
     $s \leftarrow \text{Partition}(A[l \dots r])$ 
    QuickSort( $A[l \dots s-1]$ )
    QuickSort( $A[s+1 \dots r]$ )

```

- ① Stops on encountering the first element greater than or equal to the pivot : left-to-right scan
- ② Stops on encountering the first element smaller than or equal to the pivot : right-to-left scan

① If scanning indices i and j have not crossed
i.e. $i < j$ we simply exchange $A[i]$ and $A[j]$
and resume the scans by incrementing i and
decrementing j respectively.

② If the scanning indices have crossed over i.e. $i > j$
we have partitioned the array after exchanging the pivot
with $A[j]$

③ If the scanning indices stop while pointing to the same
element i.e. $i = j$ the value they are pointing to must be
equal to p .

Combine 2 and 3

Exchange $A[j]$ with pivot whenever $i \geq j$

	5 pivot	3	1	9 i	8	2	4 j	7
	5	3	1	4	8 j	2 i	9	7
	5	3	1	4	2	8	9	7
	2 yeni pivot	3 i	1 j	4	5	8	9	7
	2	1 j	3 i	4	5	8	9	7
$j < i$ oldugunda pivot j ile yer degistirir.	1	2	3	4	5	8	9	7

Partition Algorithm:

$p \leftarrow A[l]$

$i \leftarrow l ; j \leftarrow r+1$

repeat

repeat $i \leftarrow i+1$ until $A[i] \geq p$

repeat $j \leftarrow j+1$ until $A[j] \leq p$

swap $(A[i], A[j])$;

until $i \geq j$

swap $(A[i], A[j])$ // undo last swap when $i \geq j$

swap $(A[l], A[j])$

return j

Best Case

If all the splits happen in the middle of corresponding subarrays, we will have the best case.

Recurrence Relation

$$C_{\text{Best}}(n) = 2C_{\text{Best}}(n/2) + n, n > 1$$

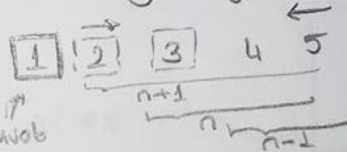
$$C_{\text{Best}}(1) = \emptyset$$

$$C_{\text{Best}}(n) \in \Theta(n \log n)$$

! Analiz yapmıdık. Merge sort'ta yapmıstık. Backward substitution yapılarak çözülür.

Worst Case

Increasing Array



$A[0 \dots n-1]$
Already sorted

$$C_{\text{Worst}}(n) = (n+1) + n + \dots + 3$$

$$= \frac{(n+1)(n+2)}{2} - 3$$

$$C_{\text{Worst}}(n) \in \Theta(n^2)$$

Average Case

$$C_{\text{avg}}(n) \approx 2n \ln n \approx 1.38 n \log n$$

Sequential Search:

0	1	2	3	4
3	5	1	4	2

$O(n)$

Binary Search:

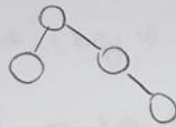
0	1	2	3	4	5	6
2	7	9	12	17	19	22

$O(\log_2 N)$

$x=7$

Binary Search Tree:

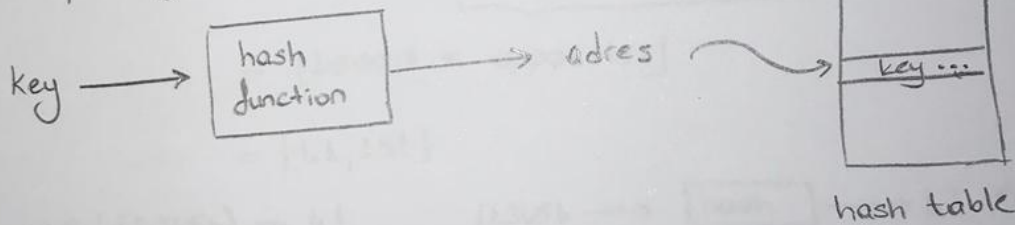
$O(n)$



denge bozulmuş olabilir.

HASHING

Unique key kullanılır.



① Hesapı kolay.

② Tutarlı olmalı.

③ Adresin olabildiğince uniform dağılımı sağlanmalı

key mod m ↗ tablo uzunluğu

$m=10 \rightarrow 2^d$

key $\rightarrow 100, 10, 40, 50 \rightarrow \emptyset$

En çok kullanılan Hash Fonksiyonları

① Division Method

$$\text{hash}(\text{key}) = \text{key} \bmod m$$

$\uparrow$
 tablo uzunluğu

$$m=11$$

$$\text{key}=25$$

$$\text{hash}(25)=3$$

0	
1	
2	
3	25...
4	
5	
6	
7	
8	
9	
10	

② Multiplication Method

$$A: \text{constant} \quad 0 < A < 1 \quad A = \frac{\sqrt{5} - 1}{2} = 0.618033$$

$$\text{hash}(\text{key}) = \lfloor m * [(\text{key} * A) \bmod 1] \rfloor$$

$$(\text{key} * A) \bmod 1 = \text{key} * A - \lfloor \text{key} * A \rfloor$$

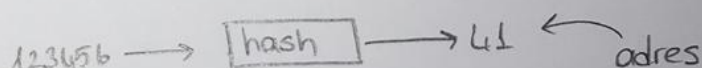
$$m = 10000 \quad k = 123456$$

$$\text{hash}(123456) = \lfloor 10000 * \underbrace{(123456 * 0.618033) \bmod 1}_{76300.0001151} \rfloor$$

$$= \lfloor 10000 * 0.0001151 \rfloor$$

$$= \lfloor 1.151 \rfloor$$

$$\text{hash}(123456) = 1$$



Birthday Paradox

$$365 \rightarrow \frac{365-1}{365} \Rightarrow \text{iki kişinin doğum gününün aynı olmama olasılığı}$$

$$365 \rightarrow \frac{365-1}{365} * \frac{365-2}{365} \Rightarrow \text{üç kişinin doğum gününün aynı olmama olasılığı}$$

$$365 \rightarrow \frac{365-1}{365} * \frac{365-2}{365} * \dots * \frac{365-k}{365} \Rightarrow k \text{ kişinin}$$

Hash map'te
table size: m

$$\frac{m-1}{m} * \frac{m-2}{m} * \dots * \frac{m-(k-1)}{m} \sim e^{-\frac{k(k-1)}{m}}$$

$\leftarrow k$ sayının
çakışmama
olasılığı

$$1 - e^{-\frac{k(k-1)}{m}} \Rightarrow k \text{ sayının çakışma olasılığı}$$

Hash collision probability

Ex: 32 bit hash values

$$M = 2^{32} \rightarrow \text{after } 250000 \text{ insert } \%100$$

$$77163 \text{ insert } \%50$$

Key

- ① Integer
- ② Floating number
- ③ String

"ELMA" "MALE"

↑ ↑
aynı sayıyı üretmemeli

Bunu engellemek için
Horner methodu kullanılır.

Horner's Method :

$$\text{hash(key)} = R * \text{str}[0] + \dots + R^L * \text{str}[L-1] \quad L \Rightarrow \text{lenght of string}$$

string'in
sayı karşılığı
bulunmalı
 $\text{str}[0] - 'A' + 1$

④ Compound Keys: 31.08.1997

$$\downarrow$$

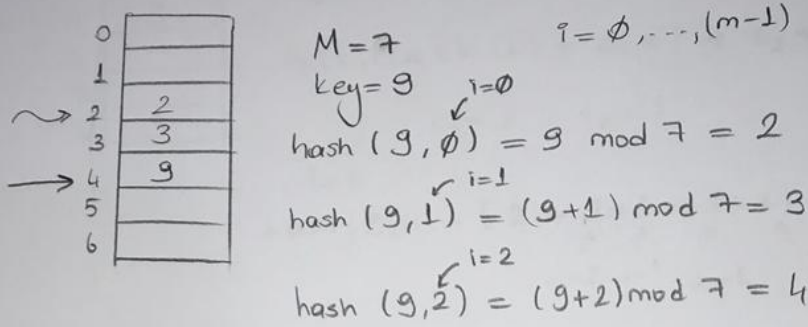
$$31 * R + 8 * R^2 + 1997 * R^3$$

Collusion Problemi Çözümü

① Open Addressing : $\rightarrow$ Table size $m \sim 2 * N$

①.1 Linear Probing :

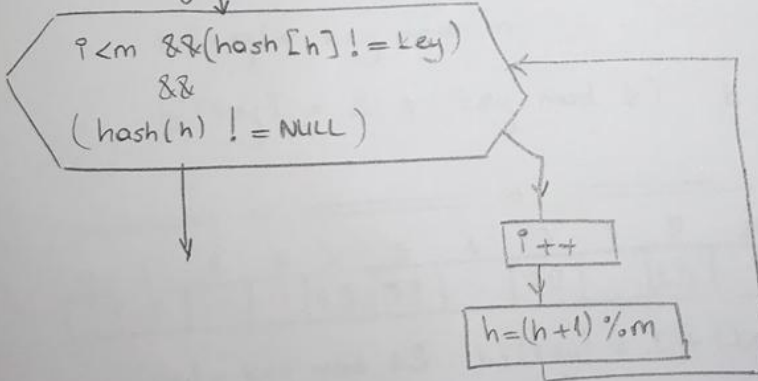
$$\text{hash}(\text{key}, i) = [h'(\text{key}) + i] \bmod m$$



Yerleştirilmek istenen yer doluysa boş yer arama işlemidir.

Search

$$h = \text{key} \bmod m$$



Delete

- ① Silinen eleman yerine bir flag konulmalıdır. Başka bir eleman ararken $\text{hash}(h) \neq \text{NULL}$ koşulunu sağlamak için bu yapılmalıdır.
- ② Silinen eleman yerini doldurmak için modu aynı olan satırlar kaydırılabilir.

1.2. Quadratic Probing

$$h(k, i) = (h'(k) + i^2) \bmod m$$

0	
1	
2	2
3	3
4	
5	
6	9

$$\text{hash}(9, 0) = 9 \bmod 7 = 2$$

$$\text{hash}(9, 1) = (2+1) \bmod 7 = 3$$

$$\text{hash}(9, 2) = (2+2^2) \bmod 7 = 6 \bmod 7 = 6$$

$$h(k, i) = (h'(k) + c_1 * i + c_2 * i^2)$$

$$c_1 = 2 \quad c_2 = 1$$

$$i = 1$$

$$\text{hash}(9, 1) = (2 + \underset{\substack{\uparrow \\ c_1=2}}{2} * \underset{\substack{\uparrow \\ i=1}}{1} + \underset{\substack{\uparrow \\ c_2=1}}{1} * \underset{\substack{\uparrow \\ i=1}}{1}) \bmod m = 5$$

1.3. Double Hashing

$$\text{hash}(\text{key}, i) = [h_1(\text{key}) + \underbrace{i * h_2(\text{key})}_{\text{adım büyüklüğü}}] \bmod m$$

$$h_1(\text{key}) = \text{key} \bmod m$$

$$h_2(\text{key}) = 1 + (\text{key} \bmod k) \quad k < m$$

0	1	2	3	4	5	6	7	8	9	10	11	12
	+9			69	58		72		14			

$$h_1(\text{key}) = \text{key} \bmod 13 \quad h_2(\text{key}) = 1 + (\text{key} \bmod 11)$$

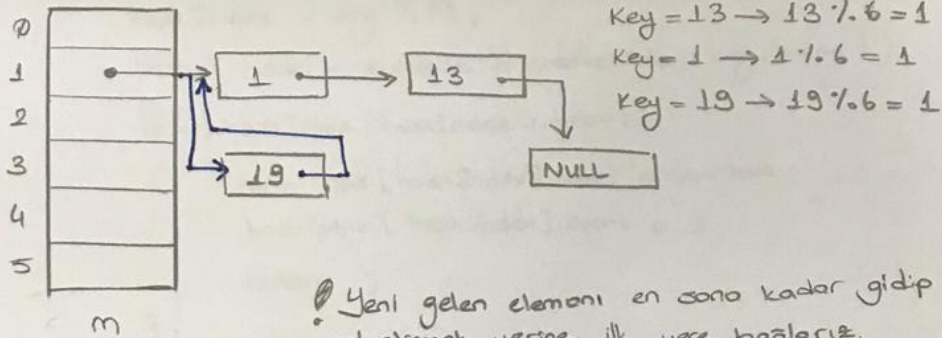
$$\text{hash}(14) = ?$$

$$\text{hash}(14, 0) = h_1(14) = 14 \bmod 13 = 1 \quad \text{dolü.}$$

$$i = 1 \Rightarrow \text{hash}(14, 1) = 1 + [1 + 14 \bmod 11] * 1 = 1 + 4 = 5 \quad \text{dolü.}$$

$$i = 2 \Rightarrow \text{hash}(14, 2) = 1 + \underbrace{[1 + 14 \bmod 11] * 2}_4 = 9 \quad \text{boş} \quad \checkmark$$

(2) Separate Chaining



$m < N$
 maximum $m = N$

m too small $\rightarrow$ chains too long

$m \sim N/5$ olursa chain'in çok uzamacağı hesaplanır.

```
struct node {
    int key;
    char name[40];
    struct node *next;
};
```

```
struct hash {
    struct node *head;
    int count;
};
```

```
struct hash *hashTable = NULL;
```

```
int main() {
    int key;
    char name[40];
    // okuma işlemleri
    hashTable = (struct hash *) calloc(M, sizeof(struct hash));
    if (hashTable == NULL) {
        printf("Yer açılmadı"); exit(0);
    }
    insertToHash(key, name);
    searchInHash(key);
    deleteFromHash(key);
}
```

```

void InsertToHash (int key, char name []) {
    int hashIndex;
    hashIndex = key % M;
    struct node *newNode = createNode (key, name);
    if (!hashTable[hashIndex].head) {
        hashTable[hashIndex].head = newNode;
        hashTable[hashIndex].count = 1;
    }
    return ;
}

newNode->next = (hashTable[hashIndex].head);
hashTable[hashIndex].head = newNode;
hashTable[hashIndex].count += 1;
return ;
}

```

```

struct node *createNode (int key, char *name) {
    struct node *newNode;
    newNode = (struct node*) malloc (sizeof (struct node));
    newNode->key = key;
    // newNode->age = age;
    strcpy (newNode->name, name);
    newNode->next = NULL;
    return newNode;
}

```

```

void searchInHash (int key) {
    int hashIndex = key % M;
    int flag = 0;
    struct node *myNode;
    myNode = hashTable[hashIndex].head;
    if (!myNode) {
        printf ("search element unavailable\n");
        return;
    }
    while (myNode != NULL && !flag) {
        if (myNode->key == key) {
            printf ("");
            myNode = myNode->next;
        }
    }
}

```

```

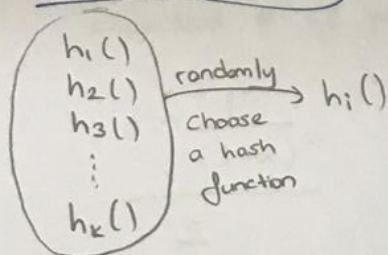
if (!flag) {
    printf ("Element not");
}
return ;
}

```



```
void deleteFromHash (int key) {  
    int hashIndex = key % M;  
    int flag = 0; struct node *tmp, *myNode;  
    myNode = (hashTable[hashIndex].head);  
    if (!myNode) {  
        printf (" Element yok");  
        return;  
    }  
    tmp = myNode;  
    while ( myNode != NULL) {  
        if (myNode->key == key) {  
            flag = 1;  
            if (myNode == hashTable[hashIndex].head) {  
                hashTable[hashIndex].head = myNode->next;  
            } else {  
                tmp->next = myNode->next;  
            }  
            hashTable[hashIndex].count --;  
            free (myNode);  
            tmp = myNode;  
            myNode = myNode->next;  
        }  
        if (flag) {  
            printf (" Deleted");  
        } else {  
            printf (" Not deleted");  
        }  
    }  
    return ;  
}
```

③ Universal Hashing



$$h_{a,b}(key) = [(a * k + b) \bmod p] \bmod m$$

$a, b, p \rightarrow$ choosen randomly

$$p=17, m=6, a=3, b=4$$

$\rightarrow$ table size is not random

$$h(8) = ((3 * 8 + 4) \bmod 17) \bmod 6 = 5$$

Open addressing : $M > N$

separate chaining : $M < N$

$$\alpha = \frac{N}{M} : \text{load factor}$$

$\alpha \Rightarrow$ genellikle 0.5 alınır.
Yer ve zaman dengesi sağlanır.

α	(average) # of probe
0.1	1.11
0.2	1.23
0.3	1.52
0.4	1.89
0.5	2.5
0.6	3.62
0.8	13.00
0.9	50.5

Unsuccessful Search

$$P(\text{first location is occupied}) = \alpha$$

$$P(\text{empty cell}) = (1 - \alpha)$$

$$P(\text{first two is occupied}) = \alpha * \alpha$$

$$P(\text{probe terminates 2 steps}) = \alpha * (1 - \alpha)$$

$\uparrow$ 1st step $\uparrow$ 2nd step

$$P(\text{probe terminates k steps}) = \alpha^{k-1} * (1 - \alpha)$$

Average # of steps :

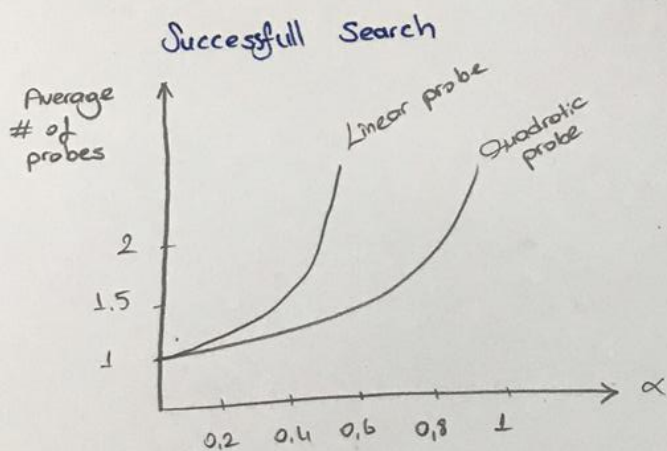
$$\sum_{k=1}^{\infty} k * \alpha^{k-1} * (1 - \alpha) \Rightarrow \text{geometric series} = \frac{1}{1 - \alpha}$$

insertion :

expected # of probes

$$\frac{1 + \frac{1}{(1-\alpha)^2}}{2}$$

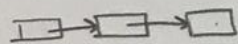
$$\alpha = 0.5 \rightarrow 2.5$$



Separate Chaining

① Unsuccessful Search

$$\alpha = \frac{N}{M}$$



worst case : Bütün elemanlar aynı adreste

$$\text{average case : } \alpha = \frac{N}{M}$$

$$\text{expected length of a probe : } \alpha = \frac{N}{M}$$

$$O(1 + \alpha)$$

search

hash function

$$\alpha = \frac{N}{M}$$

subit

$$O(1 + \alpha) = O(1)$$

keys = 64, 100, 128, 200, 300, 400, 500

tableSize = 8

$$\text{hash(key)} = x \bmod 8$$

$$64, 128, 200, 400 \rightarrow 0$$

$$100, 300, 500 \rightarrow 4$$

$$\text{gcd}(64, 100, 128, 200, 300, 400, 500, 8)$$

Dynamic Programming → Planning Richard Bellman (1950's)

Fibonacci Numbers

0 1 1 2 3 5

→ $n=0$: fibonacci(n) → 0
 $n=1$: fibonacci(n) → 1
 $n > 1$: fibonacci(n-1) + fibonacci(n-2)

```
int fibonacci(int n) {
    if (n==0)
        return 0;
    if (n==1)
        return 1;
    return fibonacci(n-1) + fibonacci(n-2);
}
```

Recurrence Relation: $\text{fibonacci}(n) = \text{fibonacci}(n-1) + \text{fibonacci}(n-2)$

0	1	2	3	4	5	6
0	1	1	2	3	5	8

fibonacci

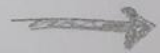
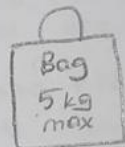
→ abandon
→ select

0-1 Knapsack Problem

Hırsız, yükte hafif pahalı ağır şeyleri almaya çalışıyor.

Brute Force: 2^n

item	weight	price
1	2	10
2	1	20
3	3	15
4	4	50



W = 5 kg max

item	weight	value
1	2	12
2	1	10
3	3	20
4	2	15

		Weight					
		0	1	2	3	4	5
item	0	0	0	0	0	0	0
	1	0	0	12	12	12	12
	2	0	10	12	22	22	22
	3	0	10	12	22	30	32
	4	0	10	15	25	30	37

max Kazanç

Amacımız:

maximize $\sum V_i$ subject to W

$P(i, j)$: maximum profit
from item: 1 to i
from weight: 1 to j

Case 1: thief takes item i

$$P(i, w) = P(i-1, w-w_i) + V_i$$

i=3 $P(3, 3) = P(2, 1) + V_3$

$w_i = 2$

esya
3kg'ya
radar
kazancı

$w_3 = 2$

tablo
ile okları
yok

Case 2: NOT takes item i

$$P(i, w) = P(i-1, w)$$

$w_1 = 2$ $P(1, 1) = 0$ $2 > 1$
 $P(1, 2) = \max \{ 12 + 0, 0 \} = 12$
 $P(1, 3) = \max \{ P(0, 1) + 12, P(0, 3) \}$
 $w_2 = 1$ $P(2, 1) = \max \{ P(1, 0) + 10, P(1, 1) \}$
 $P(2, 2) = \max \{ P(1, 2), P(1, 1) + 10 \}$
 $P(2, 3) = \max \{ P(1, 2) + 10, P(1, 3) \}$
 $P(3, 3) = \max \{ P(2, 0) + 20, P(2, 3) \}$
 $P(4, 3) = \max \{ P(3, 1) + 20, P(3, 3) \}$

$$P[i, w] = \begin{cases} P[i-1, w] & \text{if } w_i > w \\ \max \{ V_i + P[i-1, w-w_i], P[i-1, w] \} & \text{others} \end{cases}$$

initialization

$P[0, w] = 0$

$P[i, 0] = 0$

Hangi eşyaları aldığımızı nasıl buluruz?

	0	1	2	3	4	5	
0	0	0	0	0	0	0	
1	0	0	12	12	12	12	✓ $w_1=2$
2	0	10	12	22	22	22	✓ $w_2=1$
3	0	10	12	22	30	32	X
4	0	10	15	25	30	37	✓ $w_4=2$

Reconstructing Optimal Solution

Not taken: ↑

$$P[i, j] = P[i-1, j]$$

taken: ←

Edit Distance

Mesaj yazarken yanlış yazılan kelimeyi düzeltme önerileri

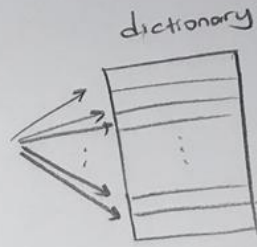
üç farklı durum var.

① sunny
Sunny

② sunny
Sunny

③ sunny
Sunnyy

sunny
↓
sunny



dist(MATHS, ARTS)

	A	R	T
M			
A			
T			
H			
S			



MATHS

-ARTS

↑ ↑

MATHS

ART-S

↑ ↑ ↑

ED[i, j]:

$x[i]$ gelenek kadarki
 $y[j]$ min mesafe

$x[1..m]$ $y[1..n]$

① copy $x[i] \rightarrow y[j]$

$$ED[i, j] = ED[i-1, j-1]$$

② change $x[i] \rightarrow y[j]$

$$ED[i, j] = ED[i-1, j-1] + \text{cost}$$

③ insert $x[i] \rightarrow -$

$$ED[i, j] = ED[i-1, j] + \text{cost}$$

④ delete $- \rightarrow y[j]$

$$ED[i, j] = ED[i, j-1] + \text{cost}$$

Recurrence Relation

$$ED[i, j] = \min \begin{cases} ED[i-1, j-1] + \phi & : \text{copy} \\ ED[i-1, j-1] + \text{cost} & : \text{change if } (x[i] \neq y[j]) \\ ED[i-1, j] + \text{cost} & : \text{insert if } (x[i] \neq y[j]) \\ ED[i, j-1] + \text{cost} & : \text{delete if } (x[i] \neq y[j]) \end{cases}$$

		A	R	T	S
	ϕ	1	2	3	4
M	1	1	2	3	4
A	2	1	2	3	4
T	3	2	2	2	3
H	4	3	3	3	3
S	5	4	4	4	3

↓
minimum
distance

Longest Common Subsequence

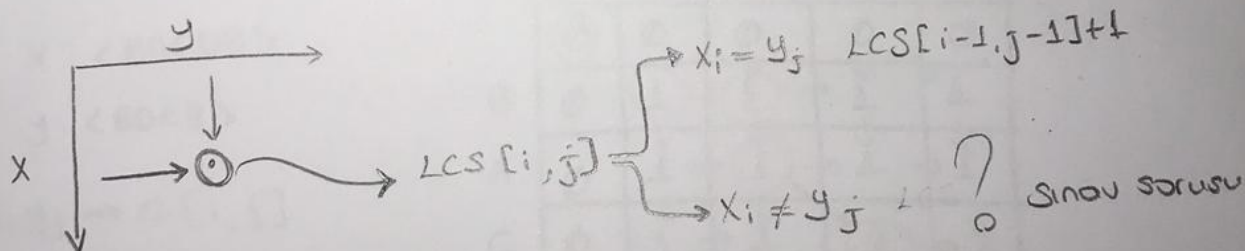
M A R K A

A R T K S A B

↓ içinde aynı
sıra ile bulundurma

X: B A C D B } LCS: B C B
y: B D C A B } B D B
BAB

DNA dizilimlerinde kullanılır.



10. hafta / 29 Kasım

Longest Common Subsequence $\langle A, C, G, T \rangle \rightarrow$ DNA dizimleri $X = ATCTGAT$ $Y = TGCATA$

) Longest Common Subsequence
 of X and $Y \rightarrow LCS(X, Y) = TCTA$
 $LCS(X, Y) = TGAT$

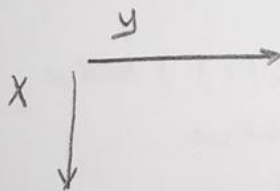
Ortak en uzun alt dizim nedir? Bu soruya cevap arayacağız.

KOZALAK Kelimesinin subsequence'lerini bulun.

$\rightarrow$
 KALV AZX
 KOLV
 KOLAV
 KALKV

$X = \langle x_1, x_2, x_3, \dots, x_m \rangle$
 $Y = \langle y_1, y_2, \dots, y_n \rangle$

} LCS $S = \langle s_1, s_2, \dots, s_k \rangle$
 $s_{i_1}, s_{i_2}, \dots, s_{i_k}$
 $i_1 < i_2 < \dots < i_k$



Matris yapısı kullanırsa
 karşılaştırma işlemi daha rahat
 olur.

 $X: \langle BACDB \rangle$ $Y: \langle BDCB \rangle$ $x_i y_j \rightarrow C[i, j]$

$x_m y_n \rightarrow$ tamamının
 uzunluğunu
 bulcaz.
 Geriye giderek
 neler olduğumuzu
 bakacağız.

	B	D	C	B
B	0	0	0	0
A	0	1	1	1
C	0	1	2	2
D	0	1	2	2
B	0	1	2	3

Initial

if $i = \emptyset \rightarrow c[\emptyset, j] = \emptyset$

if $j = \emptyset \rightarrow c[i, \emptyset] = \emptyset$

$$c[i, j] = \begin{cases} \emptyset & \text{if } i = \emptyset \text{ or } j = \emptyset \\ c[i-1, j-1] + 1 & i, j > \emptyset, x_i = y_j \\ \max \{ c[i-1, j], c[i, j-1] \} & x_i \neq y_j \end{cases}$$

ADD : $\emptyset$

SKIPX : 1

SKIPPY : 2

$x_i = y_j \rightarrow B[i, j] = \text{ADD}$

$x_i \neq y_j \rightarrow \langle [i-1, j] \rangle = c[i, j-1]$

then

$x[i]$ not in LCS

$B[i, j] = \text{SKIPX}$

else

$B[i, j] = \text{SKIPPY}$

$y[j]$ is not in LCS

while $((i \neq \emptyset) \ \&\& \ (j \neq \emptyset)) \{$

 switch $(b[i][j]) \{$

 case ADD:

 add $x[i]$ to front of LCS

$i--$;

$j--$;

 break;

 case SKIPX:

$i--$;

 break;

 case SKIPPY:

$j--$;

 break

 }

}

Vize-1 Soruları

1) Çalışma zamanı $8N^2 + 13N^{4/3} + 12N + 3$

N elemanlı dizi $\rightarrow 1$ saat

$2N$ elemanlı dizi $\rightarrow ?$ saat

Bellek ve çalışma hızının $\rightarrow \times 2$ olduğu ortamda çalıştırılacak.

ilgilendiğimiz kısım $\rightarrow N^2$

N yerine $2N$ koyarsak $(2N)^2 = 4N^2$ yani
normal şartlarda 4 saat sürerdi.

Çalışma hızı $\times 2$ olduğu için 2 saat sürecektir.

2) $3n^3 + 20n^2 + 5$ $O(n^3)$ olduğunu gösteriniz.

$$f(n) < c * g(n)$$

$$3n^3 + 20n^2 + 5 \leq c * n^3 \quad n > n_0, c > 0$$

$$\downarrow$$

$$28$$

$$n=1 \quad \checkmark \checkmark$$

$$28 \leq 28$$

3) İki algoritmanın birbiri göre hangisinin daha verimli olduğunu gösteren yöntemler nelerdir?

Time complexity

Space Complexity

- Best case
- Average Case
- Worst Case

- Theoretical Analysis
- Empirical Analysis

39

4) a) Bubble sort algoritmasının aşağıda verilen sözde kod parçasına göre karmaşıklık analizini yapın. (worst-case için)

b) Bubble sort algoritmasının best case durumu için algoritmasını $O(n)$ olacak şekilde yazınız.

Best-case $\rightarrow$ already sorted

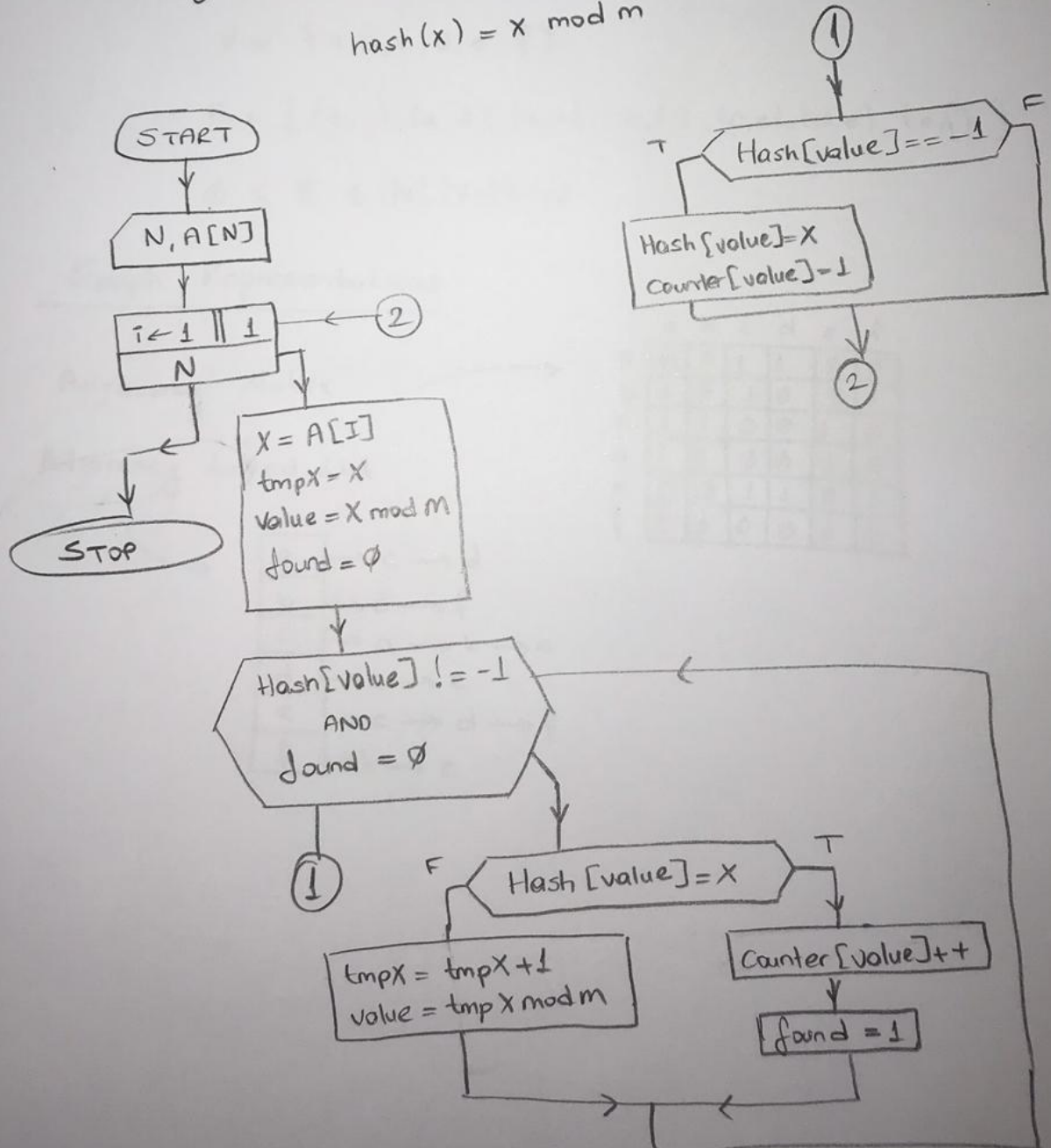
```
void bubbleSort (int N, int *dizi) {  
    int i=0, j=0, swap, tmp;  
    swap=1;  
    while ( (i < N-2) && (swap==1) ) {  
        swap=0;  
        j=0;  
        while (j < N-2-i) {  
            if (dizi[j+1] < dizi[j]) {  
                swap=1;  
                tmp = dizi[j+1];  
                dizi[j+1] = dizi[j];  
                dizi[j] = tmp;  
            }  
            j++;  
        }  
        i++;  
    }  
}
```

- 5) $n(n+5)$ ve $5000n^2$ Esit
 2^{n-1} ve 2^{n+1} Esit
 $\log_2(n)$ ve $\ln(n)$ Esit
 $1000n^2$ ve $0.001n^3$ Küçük
 $(n-5)!$ ve 2^n Büyük
 $(10n+5)\log_2 n$ ve $100n + (n+n)\log_2 n$ Esit

Verilen ifadelerin büyüme derecelerini karşılaştırarak E, B, K ifadelerini yazınız. (Sol, sağdan büyük ise B)

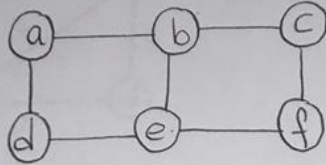
- 6) Bir A dizisindeki (N elemanlı) sayıların tekrarı kaydedilecektir. Linear probing yöntemi ile Hash'e yerleştirilmiştir.

$$\text{hash}(x) = x \bmod m$$

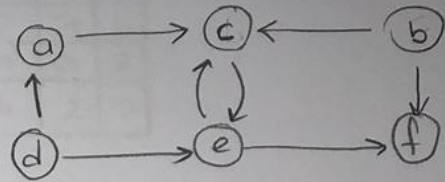


Graph

$$\rightarrow G = \langle V, E \rangle$$



Undirected



Directed

$$V = \{a, b, c, d, e, f\}$$

$$E = \{(a, c), (a, d), (b, c), (b, f), (c, e), (d, e), (e, f)\}$$

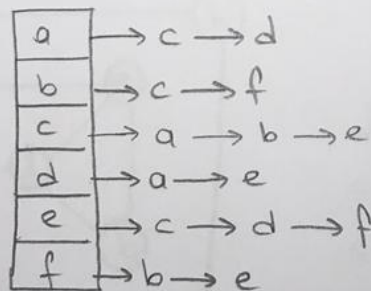
$$0 \leq E \leq (|V|(|V|-1))/2$$

Graph Representations

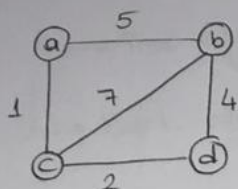
Adjacency Matrix

Adjacency Linked List

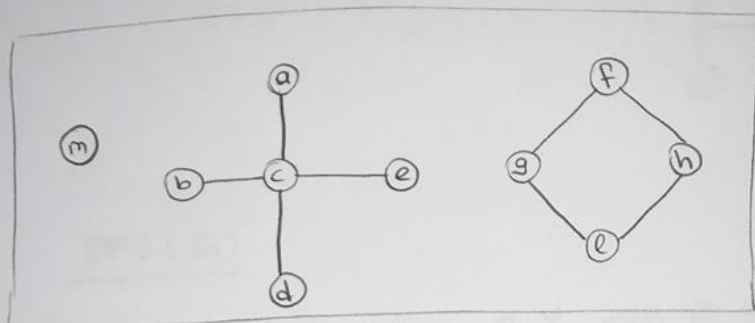
	a	b	c	d	e	f
a	0	0	1	1	0	0
b	0	0	1	0	0	1
c	1	1	0	0	1	0
d	1	0	0	0	1	0
e	0	0	1	1	0	1
f	0	1	0	0	1	0



Weighted Graph



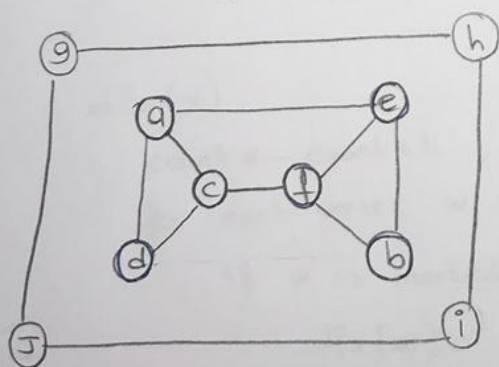
	a	b	c	d
a	0	5	1	∞
b	5	0	7	4
c	1	7	0	2
d	∞	4	2	0



connected
component
durumu

Depth-First-Search

Decrease-by-one technique

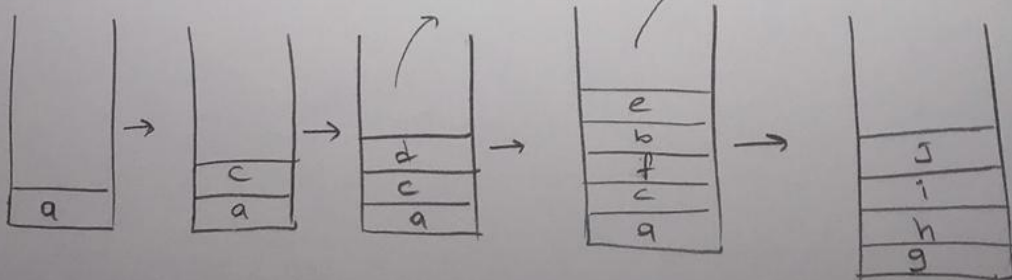


d 3,1
c 2,5
a 3,6

e 6,2
b 5,3
f 4,4

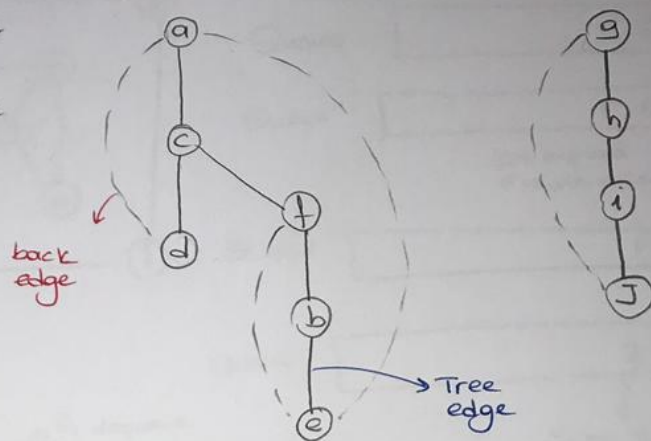
j 10,7
i 9,8
h 8,9
g 7,10

Stack $\rightarrow$ LIFO



Depth-First-Search Forest

$\rightarrow$ Tree edge
 $\rightarrow$ Back edge



DFS(G)

mark each vertex in V with $\emptyset$ as a mark of being unvisited
 $count \leftarrow \emptyset$

for each vertex v in V do

if v is marked with $\emptyset$

dfs(v)

dfs(v)

$count \leftarrow count + 1$; mark v with $count$

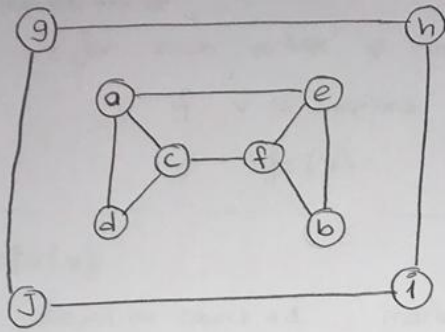
for each vertex w in V adjacent to v do

if w is marked with $\emptyset$

dfs(w)

Breadth-First-Search

Queue $\rightarrow$ FIFO



Queue EDCA

A dequeue

Queue FEDC

C dequeue

D $\rightarrow$ marked
F $\rightarrow$ unmarked

Queue FED

D dequeue

Queue BFE

Sırası ile E-F-B
dequeue olur.

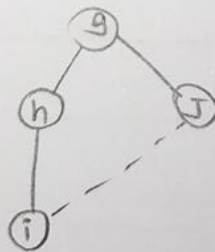
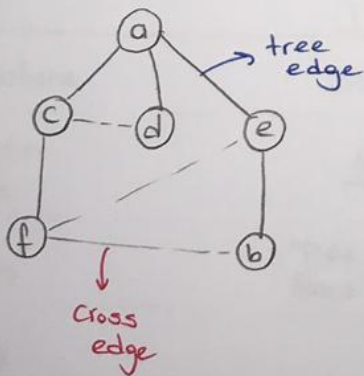
Queue JH G

G dequeue

Queue I J H

Sırası ile H-J-I dequeue olur.

Breadth-First-Search Forest



BFS(G)

mark each vertex in V with $\emptyset$ as a mark of being unvisited

count $\leftarrow \emptyset$

for each vertex v in V do

if v is marked with $\emptyset$

bfs(v)

bfs(v)

count $\leftarrow$ count + 1 ; mark v with count and
initialize a queue with v

while the queue is not empty do

for each vertex w in V adjacent to the front's vertex v do

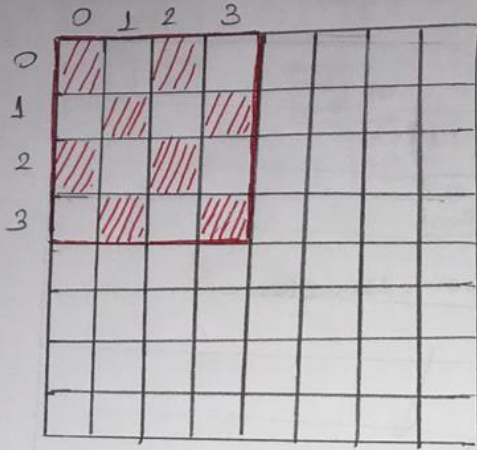
if w is marked with $\emptyset$

count $\leftarrow$ count + 1 ; mark w with count
add w to the queue

remove vertex v from the front of the queue

	DFS	vs.	BFS
Data Structure	Stack		Queue
No. of vertex orderings	2		1
Edge Types	Tree Edge Back Edge		Tree Edge Cross Edge
Applications			
Efficiency for adjacent matrix	$\Theta(V^2)$		$\Theta(V^2)$
Efficiency for adjacent linked list	$\Theta(V + E)$		$\Theta(V + E)$

8 Vezir Problemi



$$① \binom{64}{8} = \frac{64!}{8!(64-8)!} = 4\,426\,165\,368$$

② One per row

$$n^n \Rightarrow 8^8 \Rightarrow 16\,777\,216$$

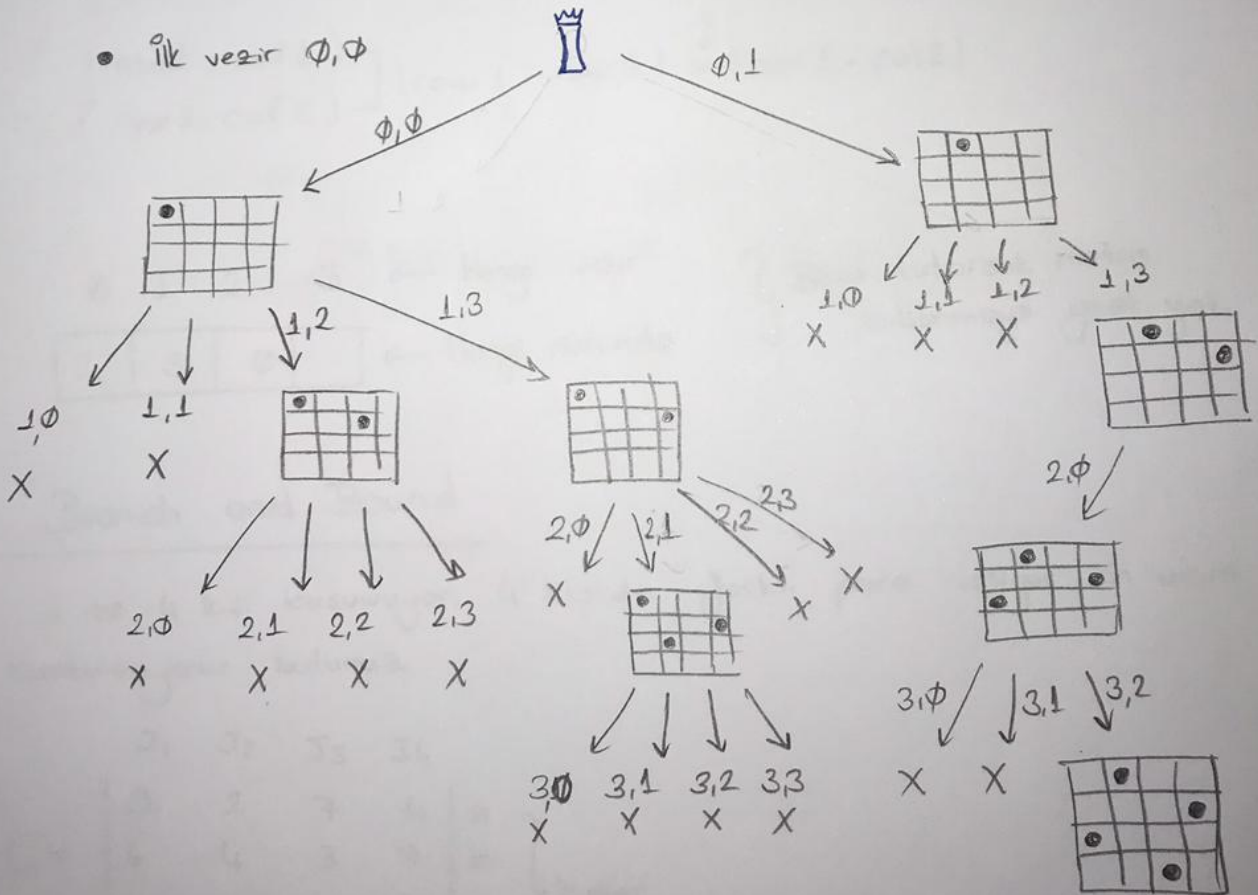
③ One per row and per column

$$n! = 8! = 40\,320$$

④ Backtracking $\rightarrow$ 2057 adım
8x8 için

! 8x8 lik zor olduğundan 4x4 lük yapılacak derste.

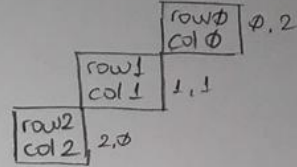
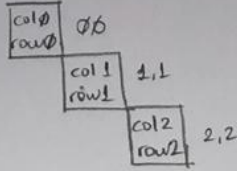
• İlk vezir $\emptyset, \emptyset$



● → kayılmış vezir
○ → kayılabilecek vezir



Diagonaldeki kontrolleri nasıl yaparız?



$$col2 - col1 = row2 - row1$$

$$\begin{aligned} \frac{col0 - col2}{2} &= \frac{row0 - row2}{2} \\ \frac{col1 - col2}{1} &= \frac{row1 - row2}{-1} \end{aligned}$$

$$|col_i - col_j| = |row_i - row_j|$$

$$\begin{aligned} (row1, col1) \\ (row2, col2) \end{aligned} \left] |row1 - row2| \stackrel{?}{=} |col1 - col2| \right.$$

0	1	2	3	← Hangi vezir
2	3	0		← Hangi sütunda

Boyle tutarsak matris kullanmaya gerek yok.

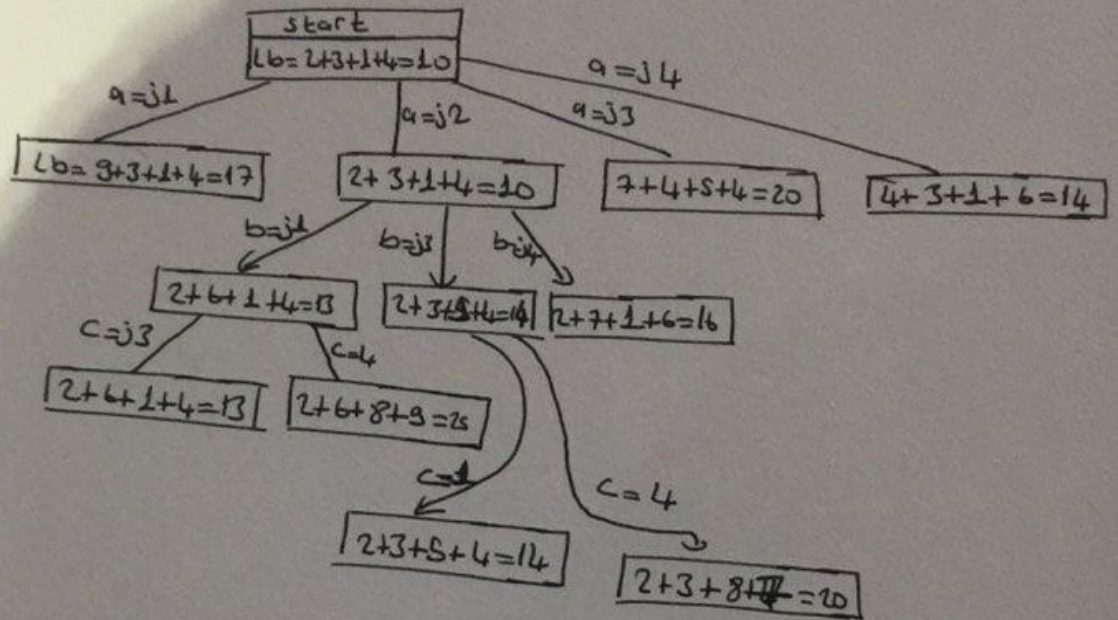
Branch and Bound

4 işe 4 kişi başvuruyor. 4 kişide farklı para istiyor. En ucuz kombinasyonu bulunuz.

$$C = \begin{array}{c|cccc|c} & J_1 & J_2 & J_3 & J_4 & \\ \hline a & 9 & 2 & 7 & 4 & \\ b & 6 & 4 & 3 & 7 & \\ c & 5 & 8 & 1 & 8 & \\ d & 7 & 6 & 9 & 4 & \end{array} \left. \vphantom{\begin{array}{c|cccc|c} & J_1 & J_2 & J_3 & J_4 & \\ \hline a & 9 & 2 & 7 & 4 & \\ b & 6 & 4 & 3 & 7 & \\ c & 5 & 8 & 1 & 8 & \\ d & 7 & 6 & 9 & 4 & \end{array}} \right\} \text{ kişiler}$$

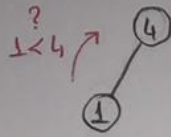
işler

$$Lb = 2 + 3 + 1 + 4 = 10 //$$



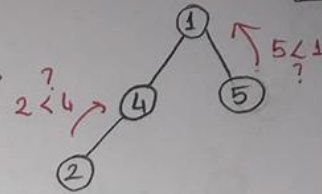
ilk olarak job fark etmeksizin en küçük sayıları topla $2+3+1+4=10$
 a kişisi için bütün joblar için yeni sonuçlar bul. $A=j3$ olduğunda
 demek oluyor ki diğerleri $j3$ 'ü alamaz. Diğerlerinin çıkışıp çıkışma-
 dığını kontrol etmeden en düşük 2. fiyatını alıyoruz. $A=j2$
 en düşük sonuç oradan devam edip aynı işlemleri b kişisi için
 yapıyoruz. b $j1$ için 13, b $j3$ seçerse $a=2$ böyle olmak zorunda.
 böyle böyle devam ediyoruz. $b=3$
 $c=5$
 $d=4$

Heapify

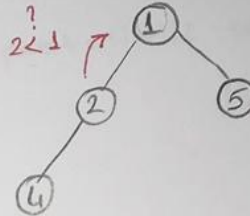


0	1	...
4	1	...

yer değiştirme yapılır.



yer değiştirilir.



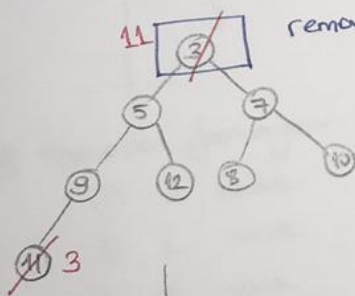
0	1	2	3	...
1	2	5	4	...

Up heap işlemi denir.

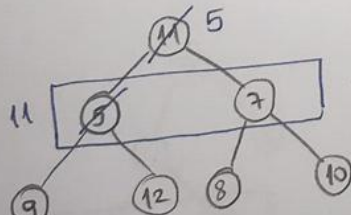
Bu işlemi insert için kullanabiliriz.

$$O(\log_2 n)$$

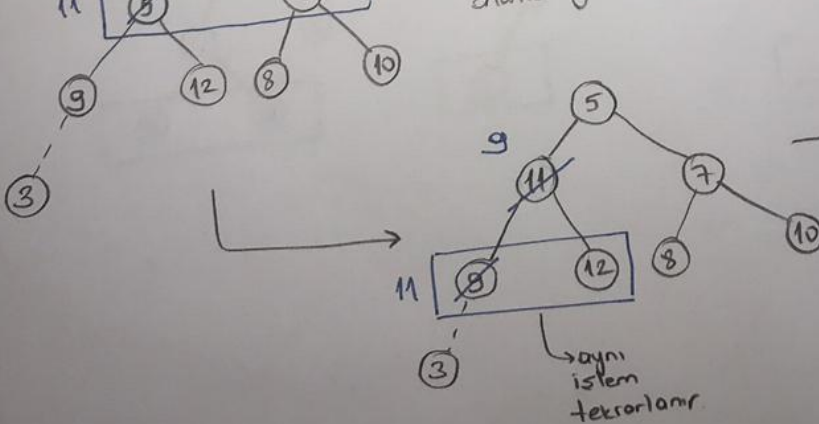
remove



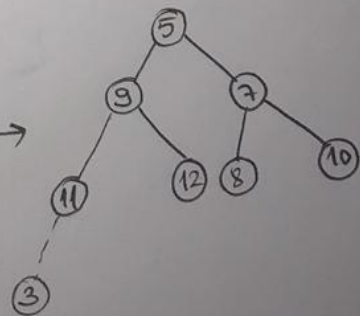
Hizmet verileni (kök değeri) en sondaki elementin yer değiştiririz. Sonra da down heap ile yerleştiririz.



→ Hangisi küçük ise onunla yer değiştirilir.



aynı işlem tekrarlanır




```
Void insert ( int newItem, int n , int pQueue[]) {
```

```
    pQueue[n] = newItem;
```

```
    int i=n, stop=0;
```

```
    n++;
```

```
    while ( (i>0) && (!stop)) {
```

```
        if ( pQueue[i] <= pQueue[(i-1)/2] ) {
```

```
            pQueue[i] ↔ pQueue[(i-1)/2];
```

```
            // swap işlemi
```

```
            i = (i-1)/2; // parent = (i-1)/2 tanımlanabilir.
```

```
        }
```

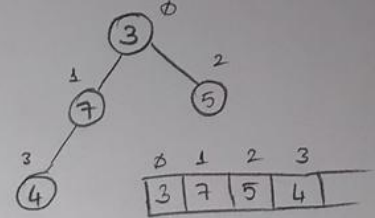
```
        else {
```

```
            stop = 1;
```

```
        }
```

```
    }
```

```
}
```



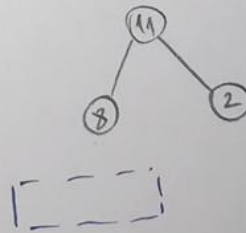
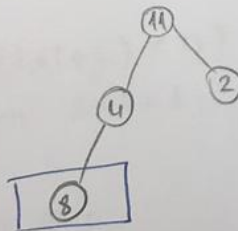
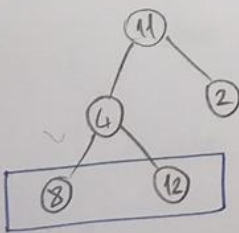
* Remove fonksiyonu için gerekli kontroller

① 2 çocuk

② 1 çocuk

③ 0 çocuk

} Bu durumlara göre işlem yapılmalıdır.



```

int remove (int n, int pQueue[]) {
    int min, i;
    if (n == 0)
        return -1;
    min = pQueue[0];
    n--;
    i = 0;
    adr = findSmallChild(i, pQueue, n);
    while ((adr > 0) && (pQueue[adr] < pQueue[i])) {
        pQueue[adr] ↔ pQueue[i]; // swap işlemi
        i = adr;
        adr = findSmallChild(i, pQueue, n);
    }
    return min;
}

```

```

int findSmallChild (int i, int pQueue[], int n) {
    if ((2 * i + 2) < n) { // 2 çocuk durumu
        if (pQueue[2 * i + 1] < pQueue[2 * i + 2])
            return 2 * i + 1;
        else
            return 2 * i + 2;
    }
    else if ((2 * i + 1) < n)
        return 2 * i + 1;
    else
        return 0;
}

```